



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۲۹ مهر

فصل دوم: لایه *Application* اسلاید ۵۹ تا ۸۷ [۱]

پروتکل *DNS*

پروتکل *DNS*

یک پروتکل سطح کاربرد است که برای هسته‌های شبکه استفاده می‌شود و پیاده‌سازی آن در *host* ها و *edge* شبکه انجام می‌شود (و نه در هسته شبکه) تا از پیچیدگی هسته جلوگیری کند.

host ها و *router* ها هم آدرس *IP* (که برای روتینگ در شبکه کاربرد دارد) دارند هم *name* عادی (که توسط انسان به کار گرفته می‌شود. مثلاً سایت *www.google.com*)، هر اسم به یک *IP*، نگاشت یا *map* شده. اسم‌ها مثل *www.google.com* از سه تا رشته یا *label* تشکیل شده.

این *mapping* از *name* به آدرس *IP* (و حتی بالعکس)، توسط سرورهای *DNS = domain name servers* انجام می‌شود. (در واقع این کار به *resolve* کردن *name* نیز معروف است) این *DNS* ها به صورت *distributed* و *hierarchical* هستند.

دلیل *centralized* نبودن آن‌ها این است که *scalable* نیست. یعنی هندل کردن یک سیستم *centralized* و تجمیع داده و ترافیک و کار *maintenance* آن، کار دشواری است. ضمناً کاربران در نقاط مختلف قرار دارند. پس اگر یک سیستم مرکزی باشد، بعضی‌ها به آن نزدیک و بعضی به آن دور هستند. همچنین چون یدونه است، از مشکل *single point of failure* نیز رنج می‌برد. برای همین بهتر است توزیع شده باشد.

در سرورهای یک دامنه، در واقع هر سرور آدرس *IP* منحصر به فرد و یک اسم منحصر به فرد معروف به *canonic* دارد و ممکن است برای راحتی کار، اسم مستعار موسوم به *alias* نیز داشته باشند.

مثلاً خود *www.google.com* نام مستعار چندین سرور وب گوگل است که هر کدام آدرس *IP* متفاوت دارند.

نقش دوم *DNS* اینجا ظاهر می‌شود

Load balancing

وقتی که از یک اسم مستعار درخواستی داریم، *DNS* لود درخواست‌ها را بین سرورهای مختلف مربوط به آدرس مستعار، تقسیم و *balance* می‌کند. درواقع پایگاه داده *DNS* دارای میلیارد ها *record* نگاشت ساده است که از آن تریلیون ها درخواست در روز می‌شود. پس باید بتواند این تعداد درخواست خواندن و پرس و جو را در روز جواب دهد. درواقع تعداد خواندن ها بیشتر از نوشتن است. پس کارای آن برای ما مهم است. و *reliability* و *security* در آن اهمیت دارند. سازمان های ارائه دهنده آن و سرور های آن به صورت توزیع شده در سراسر جهان قرار دارند.

ساختار *hierarchical DNS* در *DNS*

بعضی از سرور های *DNS*، سرور *root* هستند. از طریق آن به سرور های *tld = top level domain* دسترسی میابیم (مثل سرور های *.com* یا *.org* یا کد کشور ها مثل *.ir* و غیره) بعد از آن سرور های *authoritative* قرار دارند. مثل سرور های *google.com* یا *pbu.org* و غیره. نوع اول ارتباط و گرفتن آدرس *IP*: درواقع اول *client* به *root* درخواستی می‌دهد تا سرور *tld* می‌دهد تا آن را بدست میدهد. بعد کاربر خودش به *tld* درخواست می‌دهد تا سرور زیر شاخه آن را می‌گیرد. بعد خود کاربر به آن زیرشاخه درخواست میدهد تا در نهایت با انجام این روند به آدرس *IP* مورد نظر برسد.

DNSSEC

درواقع این بخش توسط *IETF* گسترش داده شده و روی امنیت *DNS* مثلاً یکپارچگی داده و تصدیق اصالت کار می‌کند.

سازمان *ICANN*، سرور های *root* را در سراسر جهان مدیریت می‌کند. کلاً ۱۳ تا سرور *root* به صورت *logical* هستند که البته چندین *replicate* از هر کدام در کشور های مختلف قرار دارد. هر کدام از *tld* ها هم برای خودشان سازمانی دارد که آن را مدیریت می‌کند.

local DNS

هر کدام از سازمان ها یا *ISP* ها برای خودشان، یک *local DNS* دارند که در آن بعضی از این رکورد ها را به صورت *cache* دارند. در صورتی که اون رکورد ها مثلاً وجود داشته و *out of date* نباشند، همان را برمیگردانند. (درواقع یک *TTL = time to live* دارند)

درواقع درخواست های *host* ها به *local DNS* می‌رود اول. اگر اون نمیدونست، خود *local DNS* از *root* پیگیری می‌کند تا اون رکورد را پیدا کرده و بعد از ذخیره در *cache* به *client* پاسخ می‌دهد.

انواع روش های *resolve name*

روش اول: *iterated query*

در این روش، اون *local DNS* ما درخواست را به سرور های *root* می‌فرستند. سرور های روت می‌گویند که، آدرس دقیق اون *name* را نمیدانند (مثلا آدرس *IP* سایت *www.google.com* را نمیدانند) اما آدرس *tld* آن را میداند (مثلا آدرس سرور *com* را میداند). و همان را به *local DNS* برمیگرداند در مرحله بعد، *local DNS* از *com* سوال می‌پرسد اما اون نیز آدرس دقیق را نمیداند و آدرس سروری را برمیگرداند که فکر میکند آن سرور میداند. (مثلا آدرس *google.com* را) بعد دوباره *local DNS* آدرس را از *google.com* می‌پرسد. این سرور که چند سرویس مختلف مثل وب یا ایمیل یا *FTP* و غیره دارد، و آدرس خواسته شده که مربوط به خدمات وب آن هست را نیز دارد، آدرس *IP* را برمیگرداند، بعد آن را به *host* خودش می‌دهد. (میتواند برای نیاز های بعدی کش کند)

روش دوم: *recursive query*

در این روش، *local DNS* از *root* می‌پرسد. *root* از *tld* مربوطه، یعنی *com* می‌پرسد. *tld* مربوطه از اون سرور سازمان یعنی *google.com* می‌پرسد و چون آن آدرس وب خودش را میداند، آدرس وبش را به صورت *recursive* برمیگرداند تا جواب نهایی از طریق *root* به *local DNS* برگردد. خوبی این روش این است که *node* های میانی می‌توانند رکورد های را کش کنند. بدی آن این است که روی سرور *root* لود زیادی قرار می‌گیرد.

برای کاهش لود روی سرور *root* معمولا *local DNS* ها، آدرس سرور های *tld* را نیز در خود نگه میدارند. درواقع اینجوری که فهمیدم، *name server* ها هم نگاشت ها را کش می‌کنند و هر کدام *TTL* دارند. همچنین ممکن است که نگاشت توسط *host* تغییر کرده باشد اما هنوز *TTL* به پایان نرسیده باشد، که در این صورت، اگر از سرور *DNS* با نگاشت غلط پریده شود. اطلاعات غلط داده می‌شود، اما بالاخره در درخواست های بعدی، این مشکل حل می‌شود، چون تا آن زمان *TTL* آن تمام شده است و *expire* می‌شود. درواقع به نوعی *best effort* است.

انواع رکورد های DNS

نگاشت های DNS به صورت $RR = resource\ record$ ها ذخیره می‌شوند. هر RR ، ۴ تا فیلد دارد. فیلد سوم آن *type* است که منظور تایپ شبکه است. (مثلا IN یعنی شبکه اینترنت) فیلد چهارم همان TTL است که مدت اعتبار رکورد است. فیلد اول و فیلد دوم، $name$ و $value$ مربوط به آن است. برای انواع رکورد های مختلف، دو تا فیلد اول می‌تواند متفاوت باشد

همانطور که میدانید، انواع نگاشت ها را داریم. مثلا اینکه چند دامنه را به یک آدرس IP نگاشت کنیم یا اینکه یک دامنه را برای بحث $load\ balancing$ به چند آدرس IP نگاشت کنیم. حال انواع رکورد ها را بررسی میکنیم:

- رکورد تایپ A : این نوع رکورد درواقع یک نام $hostname$ که معمولا یک اسن $canonic$ است را به یک آدرس IP مپ میکند. این نوع رکورد متداول ترین نوع رکورد است. همراه تمام رکورد های بعدی هم یک رکورد A نیز میاید.
- رکورد تایپ NS : این نوع رکورد درواقع نام مستعار دامنه را به یک سرور DNS مربوط به آن سرپرست دامنه (همان $authoritative$) مپ می‌کند.
- رکورد تایپ $CNAME$: این رکورد درواقع یک نام مستعار رو به یک نام $canonic$ آن وب سایت مپ میکند (برای www که درواقع بخش وب سایت است به کار میرود احتمالا)
- رکورد تایپ MX : این رکورد درواقع یک نامه مستعار برای ایمیل آن دامنه را به نام $canonic$ مربوط به ایمیل مپ میکند.

درواقع هر سازمان که یک دامنه ($domain\ name$) جدید ثبت کرده، باید اطلاعات آن را در سازمان مدیریت کننده آن tld معروف به $DNS\ registrar$ آن دامنه، ذخیره کند.

باید آدرس IP اون $DNS\ server$ های اولیه و ثانویه اون سرپرست دامنه هم در این $registrar$ ثبت شود به کمک یک رکورد NS و یک رکورد A .

همچنین اطلاعات وب سایت با رکورد $CNAME$ به علاوه رکورد A نیز ثبت شود.

اطلاعات ایمیل هم با دو رکورد MX و رکورد A مثل قبلی ها باید در $DNS\ server$ های سازمان $authoritative$ ثبت شود.

موضوع DNS security

دو نوع حمله متداول داریم:

DDOS attack

این نوع حملات روی *root* معمولاً جواب نمی‌دهند. چون در آنجا ترافیک فیلتر می‌شود و *local DNS* ها هم با کش کردن آدرس سرور های *tld* از این کار جلوگیری می‌کنند. اما ممکن است سرور های *tld* تحت تاثیر این حمله قرار بگیرند

spoofing attack

در این نوع حملات، مثلاً با جعل هویت، به جای سرور های *DNS* به *query* کاربران، پاسخ های غلط داده می‌شود. یا در نوع دیگر این حمله موسوم به *DNS cache poisoning* اطلاعات غلط و نگاشت ها و رکورد های غلط و جعلی در *DNS* ها ذخیره می‌شود. به کمک *DNSSEC* میتوان احراز و تصدیق هویت کرد و از این حملات به نحوی جلوگیری کرد.

معماری $P2P = peer\ to\ peer$

همانطور که قبلاً راجع به آن صحبت شد، *end user* ها به طور مستقیم به هم وصل می‌شوند، و خدمات می‌دهند و می‌گیرند. لازم نیست همیشه سرور روشن باشد. گاه گاه وصل می‌شوند و هر بار *IP* آنها تغییر می‌کند. این روش خود مقیاس پذیر است.

انواع پروتکل ها به روش *P2P*:

- برای *file sharing* پروتکل *bitTorrent*
- برای *streaming* پروتکل *KanKan*
- برای *VoIP = voice over IP protocol* پروتکل *Skype*

مقایسه $P2P$ و $client\ server$ در FTP

مدت زمان تاخیر انتقال فایل در روش $client\ server$:

$$D_{s-c} = \max\{NF/u_s, F/d_{min}\}$$

که در آن، u_s نرخ آپلود سرور به شبکه است. F حجم فایل است و چون باید سرور برای N کاربر آپلود کند، تاخیر آپلود فایل برای N نفر برابر NF/u_s است. از طرفی d_i ریت دانلود کاربر i ام از شبکه است. کمترین نرخ دانلود، بعث بیشترین تاخیر می‌شود. پس حداکثر برای تمام کاربران F/d_{min} تاخیر دانلودشان است (بیشترین تاخیر دانلود بین کاربران را گرفتیم) پس مکس بین این دو مورد (چون تقریباً همزمان هستند) برابر با حداکثر تاخیر FTP است. توجه کنید با افزایش تعداد کاربران، عبارت NF/u_s نیز به طور خطی زیاد شده و در N بزرگ غلبه می‌کند.

مدت زمان تاخیر انتقال فایل در روش $P2P$:

در این روش، هر کس که فایل را دریافت کرد، می‌تواند برای بقیه آپلود کند. پس فرمول حداکثر تاخیر FTP در آن برابریست با:

$$D_{P2P} = \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

درواقع F/u_s مدت زمان اولین آپلود فایل توسط یک سرور در شبکه است. عبارت F/d_{min} هم تاخیر دانلود توسط کاربر با کمترین نرخ دانلود است. چون بقیه کاربران هم بعد از دریافت فایل، میتوانند آن را دوباره آپلود کنند در شبکه، و نرخ هر کدام هم u_i است، پس برای NF تا فایل برای آپلود برای N نفر، میتوان از ظرفیت $u_s + \sum u_i$ استفاده کرد. در این روش، چون علاوه بر افزایش کاربران N ، ظرفیت آپلودشان هم به شبکه اضافه می‌شود یعنی u_i پس به عبارت سوم به صورت خطی زیاد نمی‌شود. درواقع اگر به اسلاید ۸۲ نگاه کنیم، حداقل زمان توزیع داده به تعداد کاربران، در $client\ server$ به صورت خطی در حال افزایش است اما در $P2P$ به ۱ میل میکند و خطی نیست.

پروتکل $bitTorrent$

این پروتکل برای انتقال فایل است. فایل‌ها را $chunk$ بندی می‌کند (مثلاً چانک‌های ۲۵۶ کیلوبایتی) به هر گروه از همتا‌ها در این شبکه که با هم چانک رد و بدل می‌کنند، $torrent$ می‌گویند. یک سرور مرکزی $tracker$ هم داریم که لیست همتا‌های شرکت کننده در $torrent$ را دارد و $track$ می‌کند.

هر کاربری که وارد می‌شود لیست را از ترکر می‌گیرد و با بعضی وصل می‌شود. که این هم‌تا‌ها همسایه‌های او هستند. *churn rate* نرخ ورود و یا خروج به *torrent* است.

فرد وارد شده بعد از دریافت فایل، میتواند به صورت خودخواهانه از *torrent* خارج شود یا به صورت نوع دوستانه در آنجا بماند و به آپلود فایل کمک کند.

درواقع همزمان با دانلود کردن، میتواند چانک‌ها را برای بقیه آپلود کنند. ممکن است چند تا از چانک‌ها را نداشته باشد. پس لیستی از چانک‌هایی را که ندارد به *peer*‌ها درخواست میدهد. بعد آن چانکی را اول می‌گیرد که *peer*‌های کمتری دارند و نادر تر است. چون ممکن است آن *peer* خاموش شود.

tit – for – tat

در این روش، فرد هر ۱۰ ثانیه، ۴ تا *peer* صمیمی تر را نگه میدارد و بقیه را بلاک می‌کند. به این عمل *choke* کردن می‌گویند. هم‌تای صمیمی، هم‌تایی است که بیشترین فایل را برای فرد فرستاده است.

از طرفی هر ۳۰ ثانیه یک بار یکی از هم‌تاهای بلاک شده را به صورت رندوم از بلاک درمیاورد و برایش فایل می‌فرستد که به ای عمل *unchoke* می‌گویند. ممکن است با این کار، آن هم‌تا نیز برایش فایل بفرستد و اینگونه صمیمی شود. درواقع ایجاد انگیزه می‌کند تا نرخ آپلود بیشتری داشته باشند. و شرکای بهتری داشته باشند.

مراجع

[1] “ch2-v8-slide-59-to-87,” <https://docs.google.com/presentation/>, 2022.